SOFTWARE ARCHITECTURE DESIGN

MODELING SOFTWARE ARCHITECTURES

HALUK ALTUNEL

PHD, PMP, CLP, PMC, PLC, PEC

'NO MATTER HOW GOOD THE TEAM OR HOW EFFICIENT THE METHODOLOGY, IF WE'RE NOT SOLVING THE RIGHT PROBLEM, THE PROJECT FAILS'

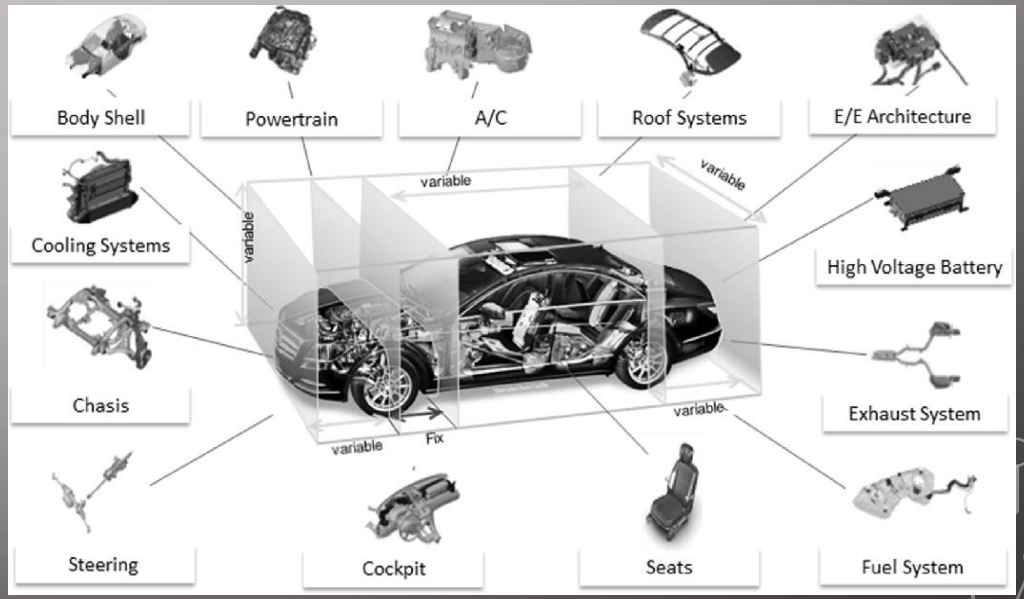
WOODY WILLIAMS

WHY?

Why we need software architecture?

ABSTRACTION

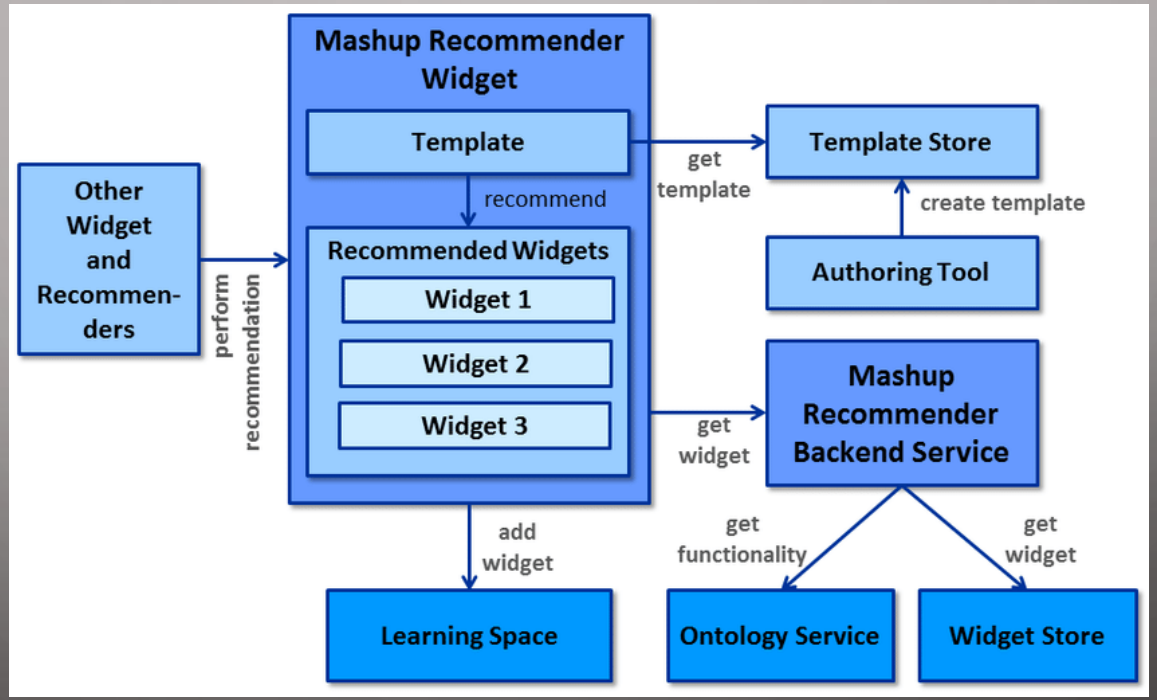
- **Focus on relevant parts**
- **High level**
- **Not every detail**
- **Copes with complexity**
- **Understanding of the system**



Toepfer, F., & Naumann, T. (2016). MANAGEMENT OF VEHICLE ARCHITECTURE PARAMETERS.

ABSTRACTION

- **Example**



Nussbaumer, A., Dahrendorf, D., Schmitz, H. C., Kravčík, M., Berthold, M., & Albert, D. (2014). Recommender and guidance strategies for creating personal mashup learning environments. Computer Science and Information Systems, 11(1), 321-342.

COMMUNICATION

- **Stakeholder communication**
- **Who are the stakeholders?**
- **Common medium**
- **Understanding**

EVOLUTION/CHANGE

- **Software system evolves and changes**
- **Evolution types:**
  - **Local**
  - **Non-local**
  - **Architectural**
- **Help dealing with evolution and scope of it.**
- **Earliest design decisions**

GUIDANCE OF DEVELOPMENT

- **Software development life cycle**
- **Defines main subsystems, interactions etc.**
- **Guide software development process**

ORGANIZATION OF DEVELOPMENT

- **Influences organizational structure**
- **Help decomposing system into subsystems and assigning teams**

REUSABILITY

- **Reuse of subsystems/components in other products & projects**

SOFTWARE ARCHITECTURE

- **The set of structures necessary to understand the system, software elements, relations among them, and properties.**
- **Make up a system, and how the network works from a general sense.**
  - **The network's working state (interactions between software elements and their contexts)**
  - **Functionality**
  - **Non-functional requirements**

SOFTWARE ARCHITECTURE

- **Fundamental attributes of software architecture:**
  - **Usable – That is software developers should be able to code functionality into it.**
  - **Realizable – That is there exists a medium such that software with this architecture can be deployed.**
  - **Maintainable – That is further work on software should adhere to the architecture**
- **Can be added more like secure, interoperable, etc.**

SOFTWARE ARCHITECTURE

- **Software architecture:**
  - **As a product (outcome)**
  - **As a process**
  - **With inputs & constraints & assumptions**
  - **Series of design decisions among alternatives**

SOFTWARE ARCHITECTURE

- **The most basic decision making model:**
  - **Problem formulation**
  - **Presentation of alternatives**
  - **Evaluation of attributes relevant to problem**
  - **Mechanism to eliminate worse alternatives**

SOFTWARE ARCHITECTURE

- **Set a common communications protocol for architectural information:**
  - **The protocols do not include non-architectural information.**
  - **Architectural information is usually about the important design decisions that are few in number.**
  - **Such information is short and not so detailed.**
  - **However, this is not a rule. Sometimes the rationale for a design decision may involve more work.**

SOFTWARE ARCHITECTURE

- **3 questions about architecture:**
  - **What specific information should be included?**
  - **How it is used?**
  - **Why it includes that information?**

- **Elements+Form+Rationale**

SOFTWARE ARCHITECTURE

- **IEEE 1471 summarizes:**
  - **Fundamental organization of a system embodied in its components, (Elements)**
  - **Their relations to each other, and to the environment, (Form)**
  - **Principles guiding its design and evolution. (Rationale)**

SOFTWARE ARCHITECTURE

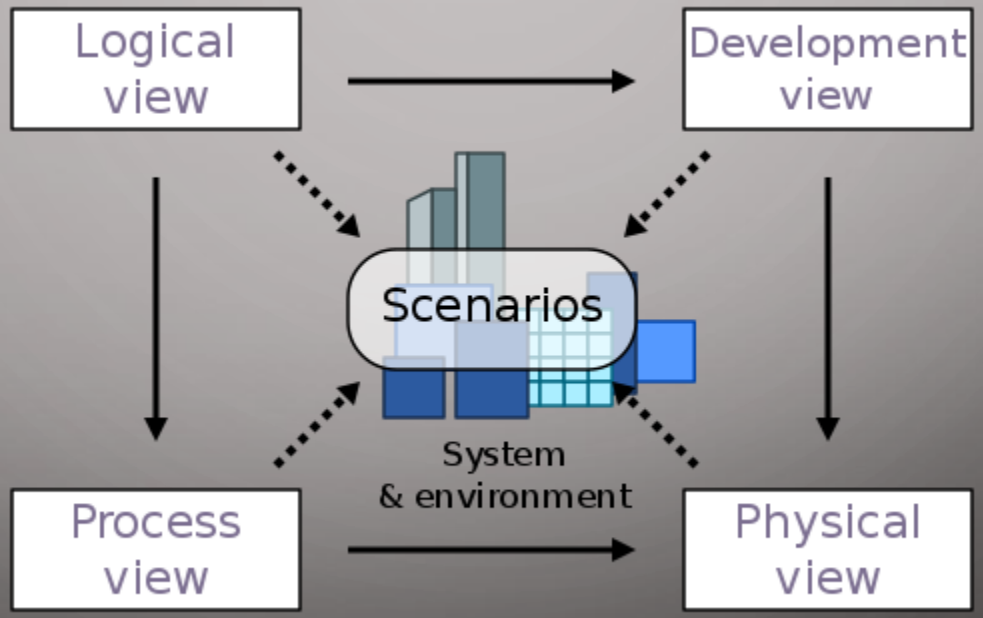
- **Terms about architectural design process:**
  - **Specification – An architecture communicated with a formal language, and formal models and the rationale expressed in a language of choice.**
  - **Representation – Representation is any useful model or abstraction of a thing related to the architecture. Representations often leave out details and some qualitative aspects of things but they are useful.**
  - **Description – Any organized collection of formal and informal specification and representation of an architecture that captures the essential aspects of its elements, forms and rationale.**
  - **Documentation – That makes the architectural description accessible through the tangibility of the documents.**

SOFTWARE ARCHITECTURE DOCUMENTATION

- **Organizational guidelines or standards for all types of documentation.**
  - **Some standards gain acceptance. IEEE standards 1471 and 42010, or IBM's «4+1» approach are among these.**
  - **Sometimes the standards are about the process. Agile documentation practices and observed practices of open source projects are de-facto standards about the process.**

SOFTWARE ARCHITECTURE DOCUMENTATION

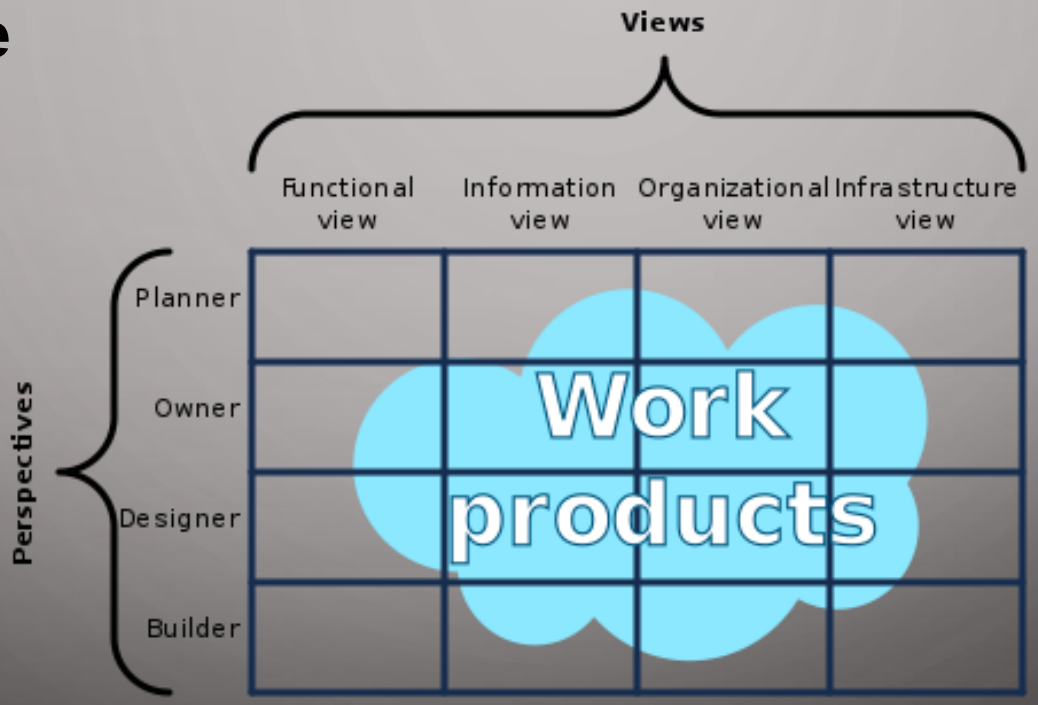
- **Example**



Kruchten, P. B. (1995). The 4+ 1 view model of architecture. IEEE software, 12(6), 42-50.

SOFTWARE ARCHITECTURE DOCUMENTATION

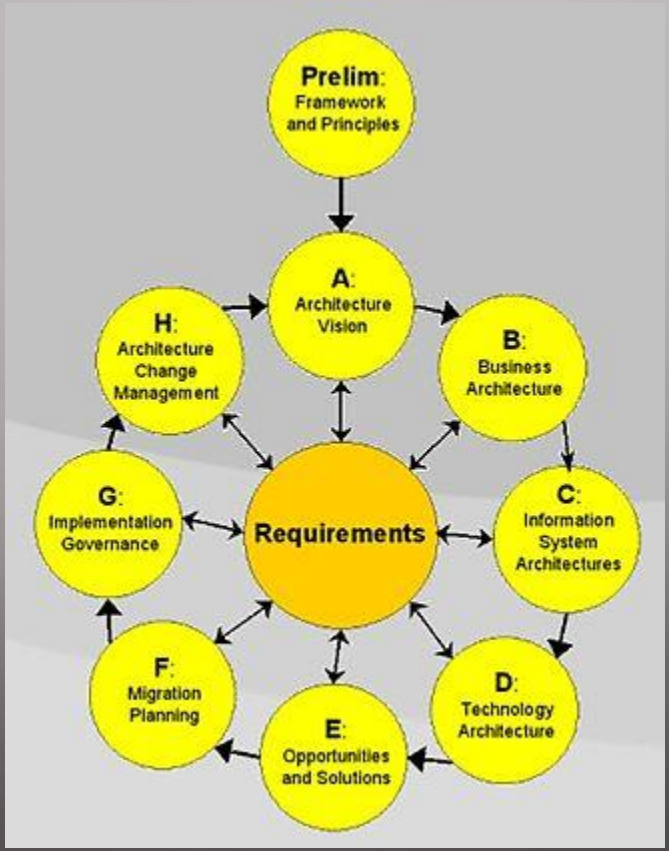
- **Example**



Treasury Enterprise Architecture Framework (TEAF)

SOFTWARE ARCHITECTURE DOCUMENTATION

- **Example**



The Open Group Architecture Framework (TOGAF)

SOFTWARE ARCHITECTURE DOCUMENTATION

Did you ever prepare any document for your software architecture?

SOFTWARE ARCHITECTURE DOCUMENTATION

- **Design oriented questions:**
  - **What does the stakeholders need?**
  - **Record design decisions according to the view(point)s of stakeholders.**
- **Document oriented questions:**
  - **Check the documentation to see if it will satisfy the stakeholders.**
  - **Package the information in a way that can be understood by the stakeholders.**

SOFTWARE ARCHITECTURE DOCUMENTATION

- **Context and development process may affect the architecture documentation:**
  - **What if context changes?**
  - **What if cannot be implemented exactly as envisioned?**
  - **What if self-aware components (Artificial Intelligences) that adapt themselves and change their architectural descriptions?**
  - **What if system that is redesigned and rebuilt very rapidly?**

VIEWS AND BEYOND METHOD

- **No single view can completely represent the architecture.**
- **Each view includes**
  - **Graphical representation for overview**
  - **An index or catalog**
  - **Specifications**
  - **Variability guide explaining contingencies and how to tailor**
  - **Rationale and design information**
- **Documentation that applies to all views (fundamental truths) include**
  - **Constraints and rationale for overall architecture**
  - **How views relate to each other.**

VIEWS AND BEYOND METHOD

- **When architecture changes to often than you can document, then:**
  - **Document what is true and will always be true.**
  - **Document the way architecture is allowed to change.**
  - **Make the system auto-generate architecture-of-the-moment documentation.**

ARCHITECTURAL STYLES

What is your architectural style?

ARCHITECTURAL STYLES

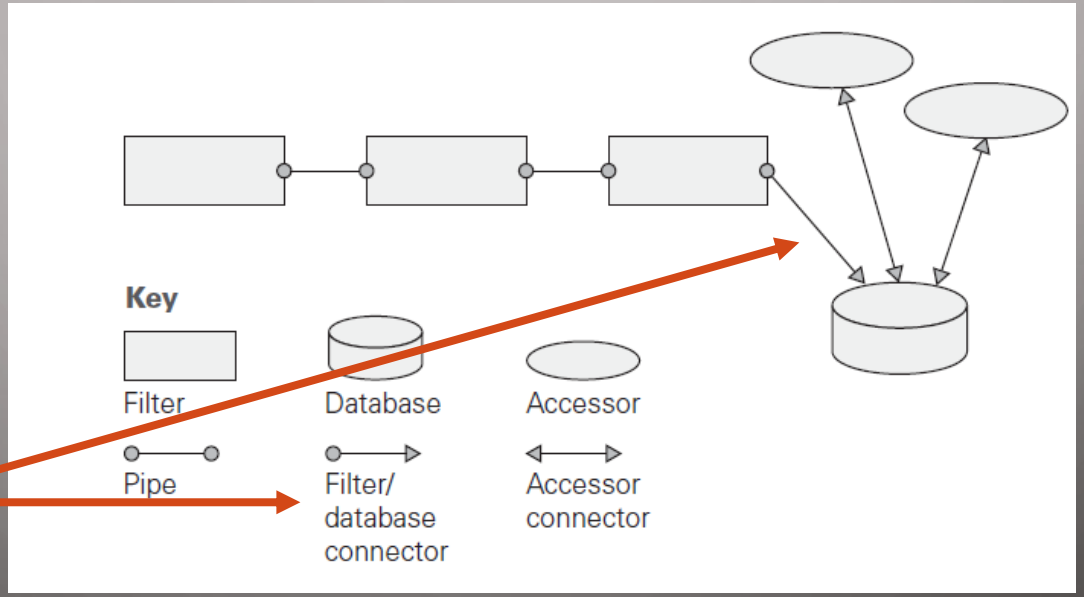
- **An architectural style is a specialization of element and relation types together with a set of constraints on how they can be used.**
- **Styles are about approaches and methodology, therefore more generic.**
- **An architectural pattern expresses a fundamental organizational schema for software systems.**
- **Patterns involve a well defined recurring problem, and a well defined schema of a solution.**
- **Patterns are more specific.**

VIEWS AND BEYOND METHOD

- **3 major categories of styles**
  - **Structured in terms of implementation units – Module Styles**
  - **Structured in terms of elements with behaviour and interactions between elements – Component and Connector Styles**
  - **Related to non-software structures in its environment – Allocation Styles**
- **When an architectural style is applied to a system, we get a view.**

VIEWS AND BEYOND METHOD

- **Example**



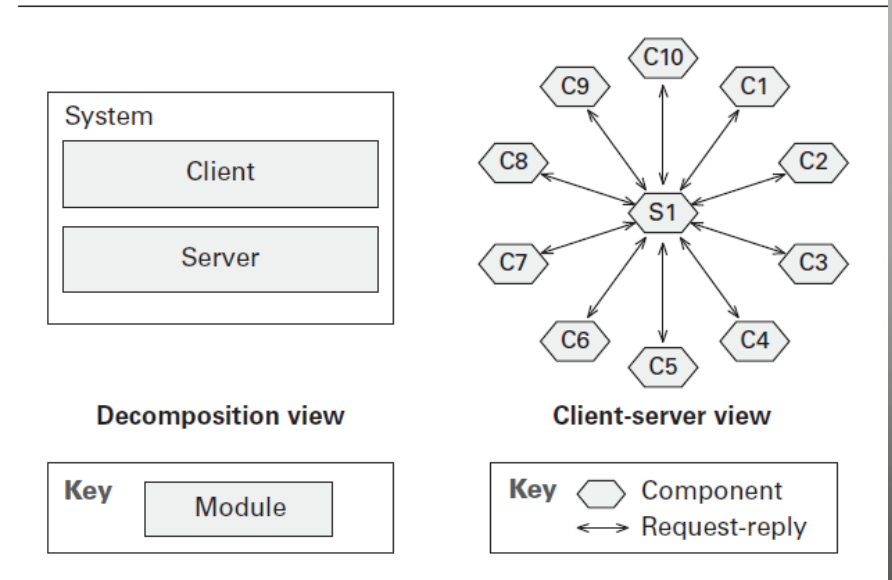
Bridging Element

Pipe and filter and shared data styles, Figure P.2 from Textbook

VIEWS AND BEYOND METHOD

- **Example**

What is a module?



What is a component?

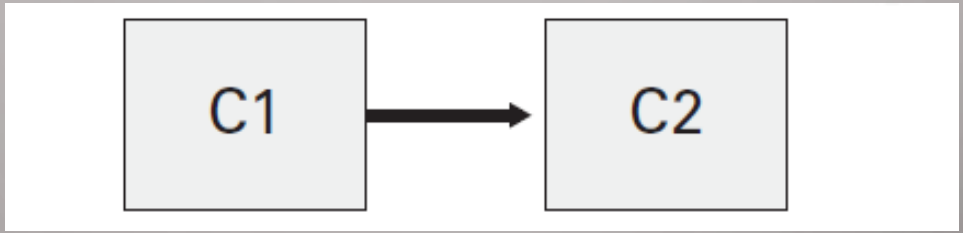
Figure P.5 from Textbook

VIEWS AND BEYOND METHOD

- **Seven rules for sound documentation:**
  1. **Write documentation from the reader's point of view**
  2. **Avoid unnecessary repetition**
  3. **Avoid ambiguity**
  4. **Use a standard organization**
  5. **Record rationale**
  6. **Keep documentation current but not too current**
  7. **Review documentation for fitness of purpose**

VIEWS AND BEYOND METHOD

What does arrow mean?



- **Possibilities:**
  - **C1 calls C2**
  - **Data flows from C1 to C2**
  - **C1 instantiates C2**
  - **C1 sends a message to C2**
  - **C1 is a subtype of C2**
  - **C2 is a data repository and C1 is writing data to C2**
  - **C2 is reading data from C1**

ARCHITECTURAL LANGUAGES

- **Architectural languages:**
  - **Fictional examples of language use that are simpler than our own everyday language, but when used in their very specific area of software architectures are effective means of communication.**
  - **Languages where meaning of symbols are strictly separated so that meaning consists of only the language but not the actions or context.**

ARCHITECTURAL LANGUAGES

- **Informal Notations:**
  - **General purpose diagrams and natural languages.**
- **Semiformal Notations:**
  - **Standardized notation but not provide a complete semantic treatment. Example: Unified Modeling Language (UML)**
- **Formal Notations:**
  - **Standardized notation with precise semantics. Example: Architecture Description Languages (ADLs).**

ARCHITECTURAL LANGUAGES

- **Architectural Description Languages (ADLs):**
  - **Usually made up of graphical modeling notations.**
  - **Some ADLs are better used for formal specifications, some for representations.**
  - **Not all modeling notations are architectural languages.**

ARCHITECTURAL LANGUAGES

- **Architectural Description Languages (ADLs) can:**
  - **Components**
  - **Connections**
  - **Hierarchical Composition**
  - **Communication**
  - **Formal Models**
  - **Computation Paradigms**
  - **Semantics, Constraints, and Non-Functional Properties**
  - **Tool Support**
  - **Modelling, Analysis, Evaluation, Verification and Validation**

ARCHITECTURAL LANGUAGES

- **Architectural Description Languages (ADLs) try:**
  - **Representation**
  - **Design Process Support**
  - **Static and Dynamic Analysis**
  - **Specification-Time and Execution-Time Evolution**
  - **Refinement**
  - **Traceability**
  - **Simulation**

ARCHITECTURAL LANGUAGES

- **Architectural Description Languages (ADLs) divided:**
  - **Industry develops more expressive and usable languages that are easily extended to define extra-functional properties, but at the cost of formalism.**
  - **Academia develops heavily formalized languages that better capture structure, styles, formalization across levels of detail, and highly dynamic behavior, but at the cost of difficulty in use.**

ARCHITECTURAL LANGUAGES

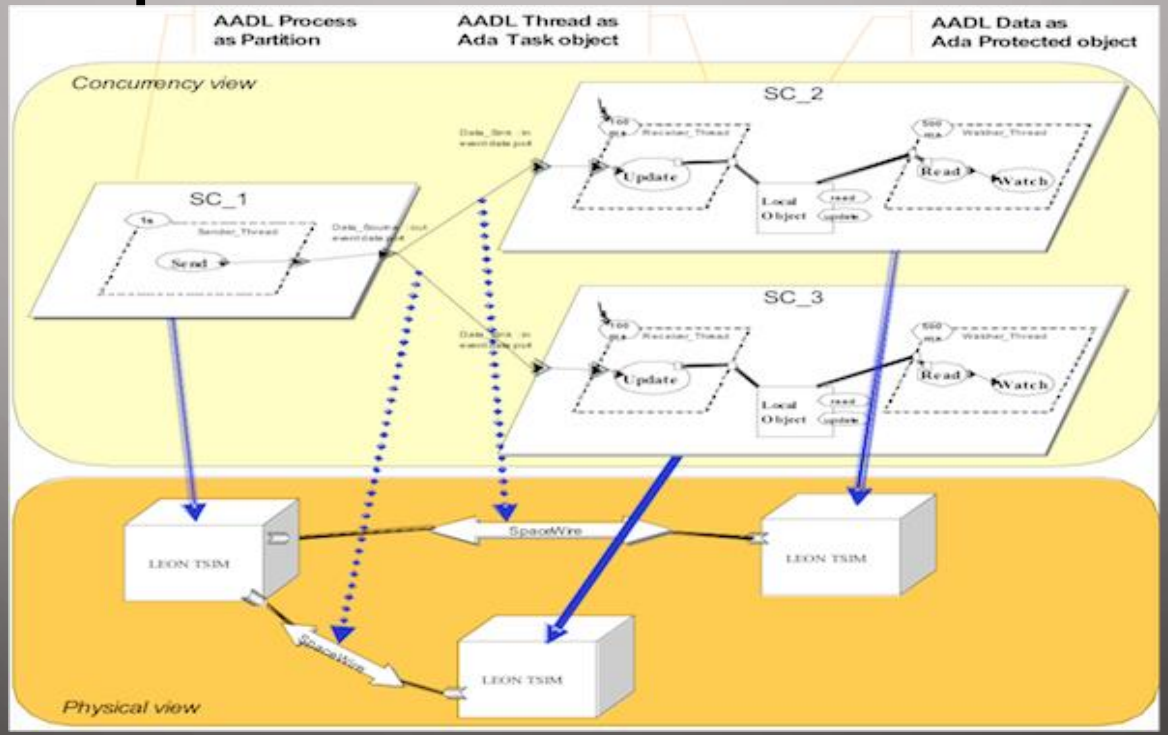
- **Architectural Description Languages (ADLs) deficient:**
  - **Restrictive assumptions which become inappropriate in many domains, hence restricting usefulness**
  - **Orientation toward a single architectural view**
  - **Lack of tools (UML, code reverse engineering, etc.)**
  - **Lack of domain specific tools, and entity libraries to assist architects in domain specific projects**
  - **Mismatch on assumed methodology and actual methodology of architectural design process.**

ARCHITECTURAL LANGUAGES

- **ADL Examples**
  - **Acme (developed by CMU)**
  - **AADL (standardized by the SAE)**
  - **C2 (developed by UCI)**
  - **SBC-ADL (developed by National Sun Yat-Sen University)**
  - **Darwin (developed by Imperial College London)**
  - **Wright (developed by CMU)**
  - **….**

ARCHITECTURAL LANGUAGES

- **AADL Example**



complete space system http://www.openaadl.org

Thank you

https://www.linkedin.com/in/halukaltunel/
https://hbrturkiye.com/yazar/haluk-altunel
http://yeniprojeyonetimi.blogspot.com/
altunel@bilkent.edu.tr